Data Structures and Algorithms

LECTURE 05: SORTING ALGORITHMS

greenwich.edu.vn

UNIVERSITY OF GREENWICH

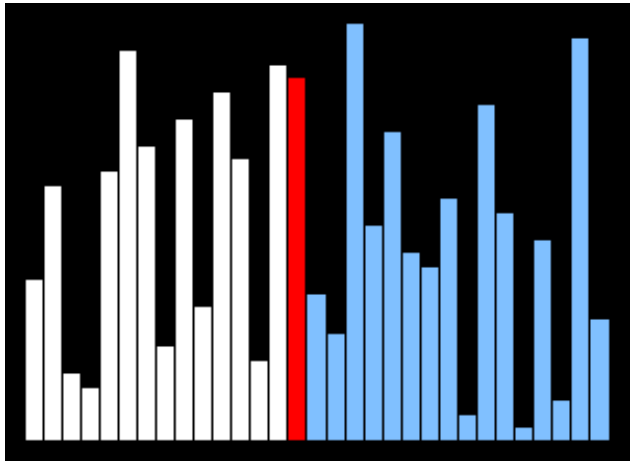
Alliance with FPT Education

Contents

- Simple Sorting Algorithms
  - Selection and Bubble Sort
- Shuffling
- Advanced Sorting Algorithms
  - QuickSort, MergeSort, BucketSort

What is a Sorting Algorithm?

- **Sorting algorithm**
  - An algorithm that rearranges elements in a list
    - In non-decreasing order
  - Elements must be **comparable**
- More formally
  - The **input** is a sequence / list of elements
  - The **output** is an rearrangement / **permutation** of elements
    - In non-decreasing order



Sorting – Example

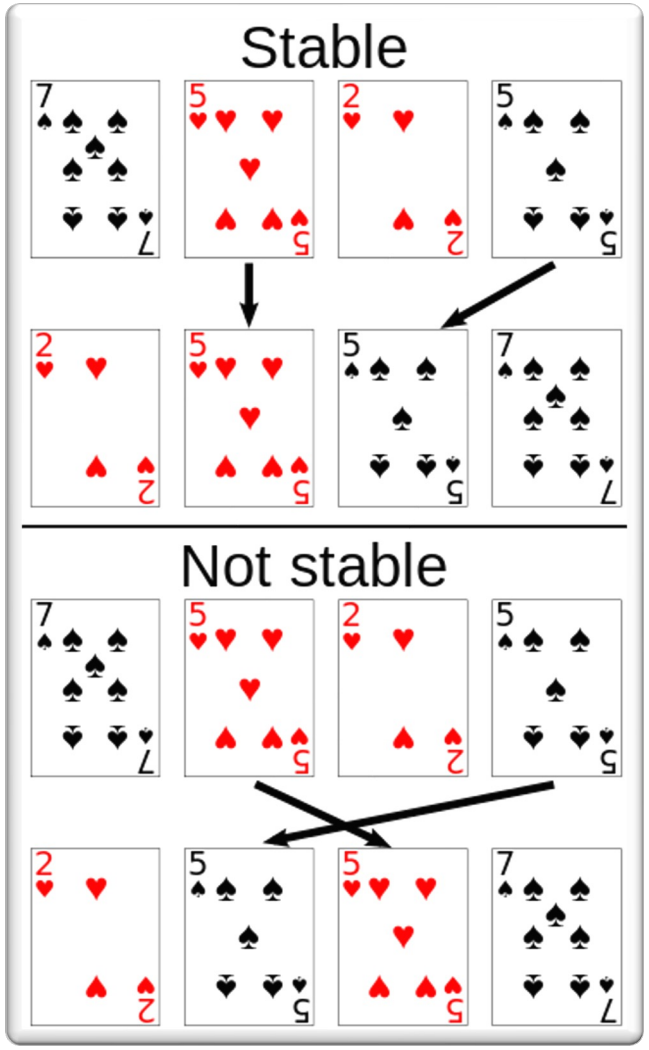
- Efficient sorting algorithms are important for:
  - Producing human-readable output
  - Canonicalizing data – making data uniquely arranged
  - In conjunction with other algorithms, like binary searching
- Example of sorting:

Sorting Algorithms: Classification

- Sorting algorithms are often classified by:
  - Computational **complexity** and memory usage
    - Worst, average and best case behavior
  - **Recursive** / non-recursive
  - **Stability** – stable / unstable
  - **Comparison-based** sort / non-comparison based
  - Sorting **method**: insertion, exchange (bubble sort and quicksort), selection (heapsort), merging, serial / parallel, etc.

Stability of Sorting

- **Stable** sorting algorithms
  - Maintain the order of equal elements
  - If two items compare as equal, their relative order is preserved
- **Unstable** sorting algorithms
  - Rearrange the equal elements in unpredictable order
- Often **different elements** have **same key** used for equality comparing



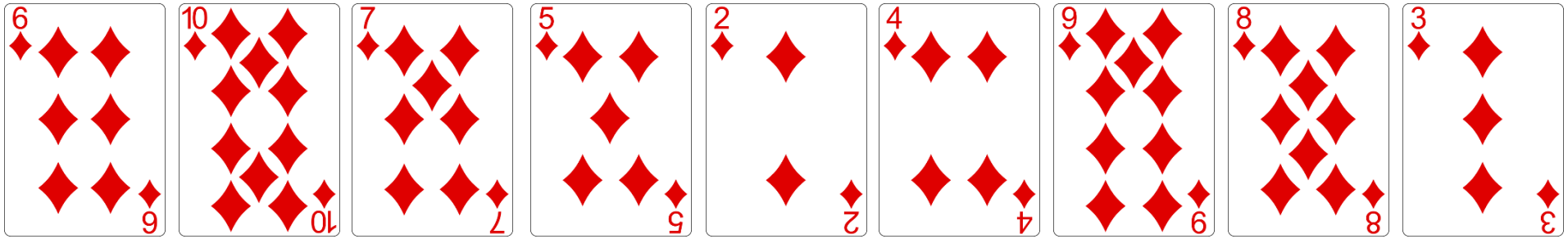
Selection Sort

- **Selection sort** – simple, but inefficient algorithm
  - Swap the first with the min element on the right, then the second, etc.
  - Memory: O(1)
  - Stable: No
  - Method: Selection

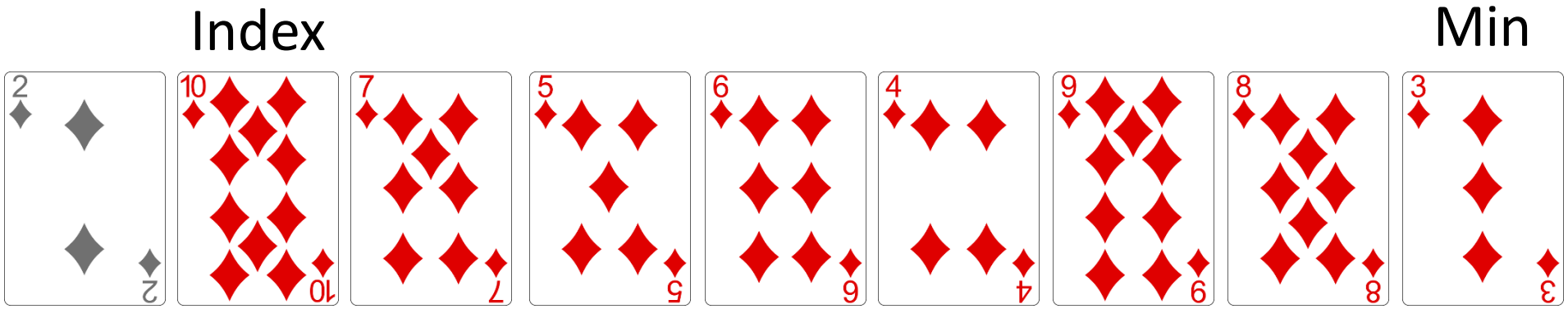
Selection Sort Visualization

Index

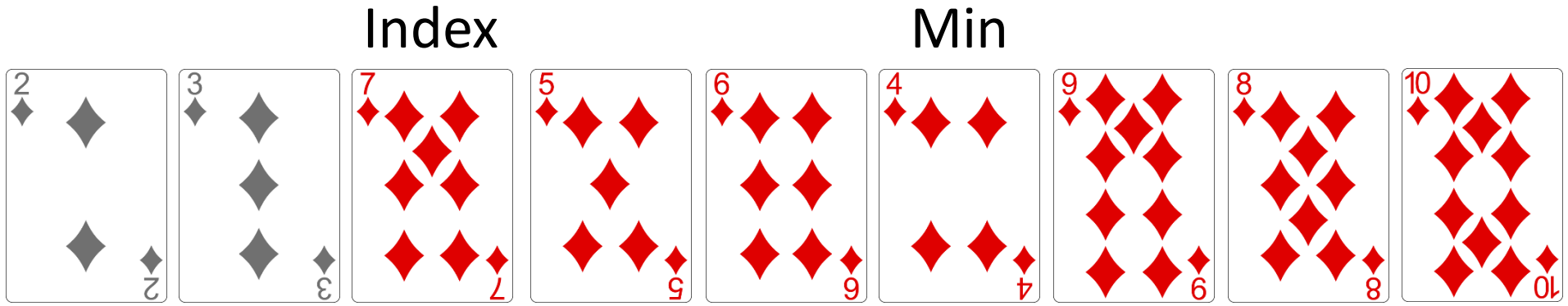
Min



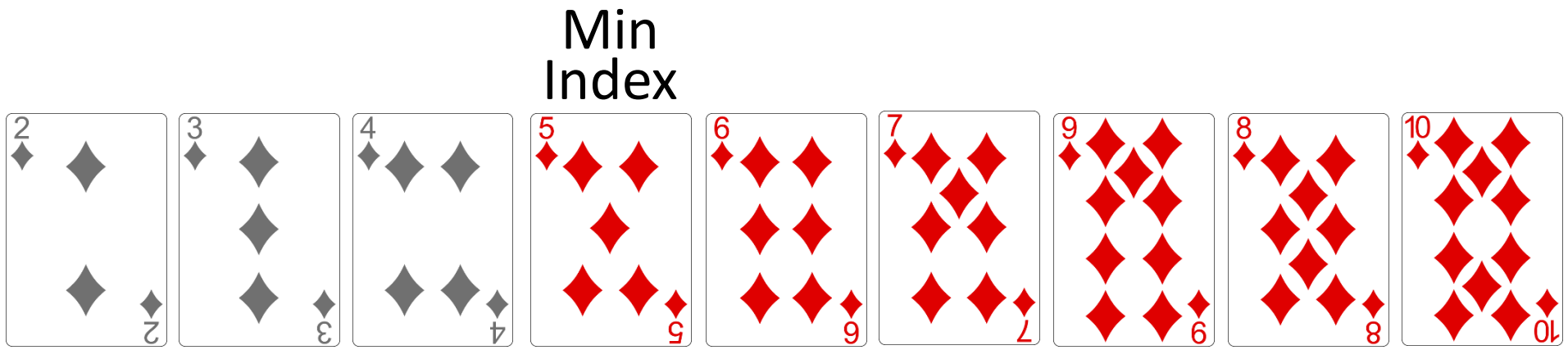
Selection Sort Visualization



Selection Sort Visualization



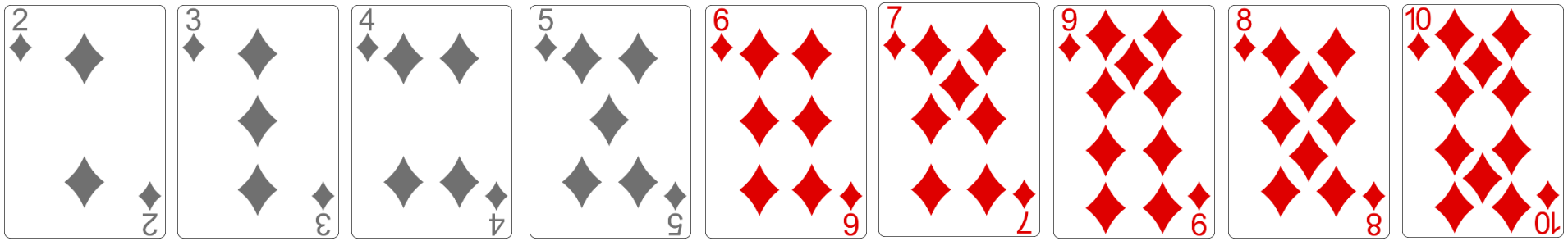
Selection Sort Visualization



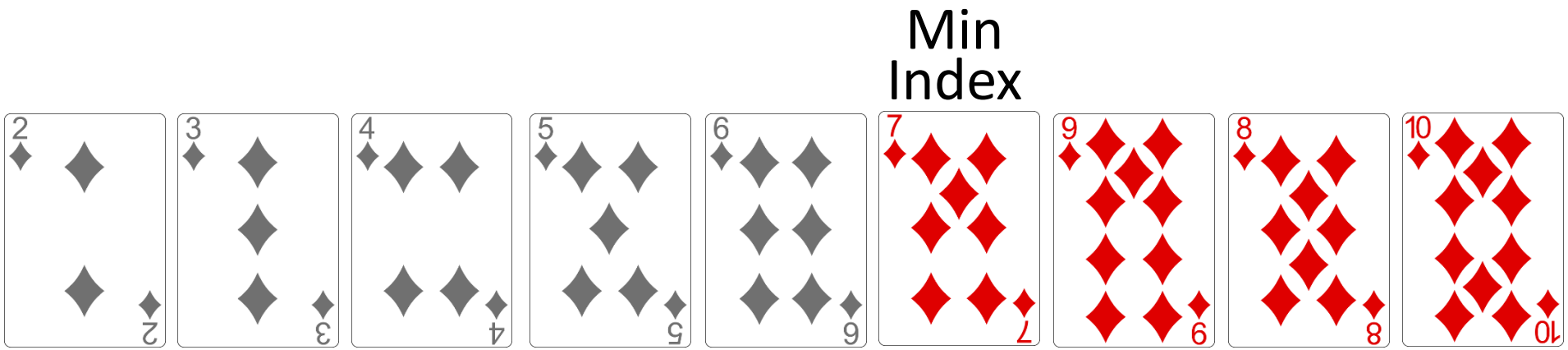
Min
Index

Selection Sort Visualization

Min
Index

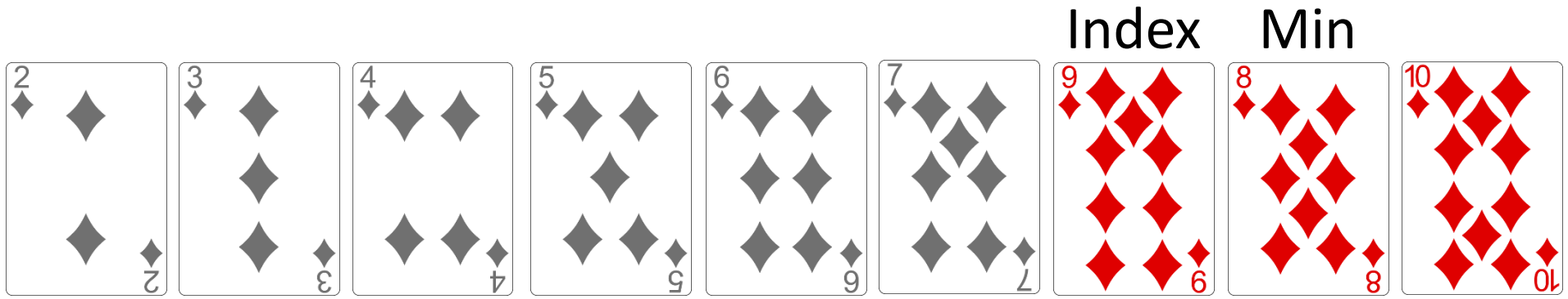


Selection Sort Visualization

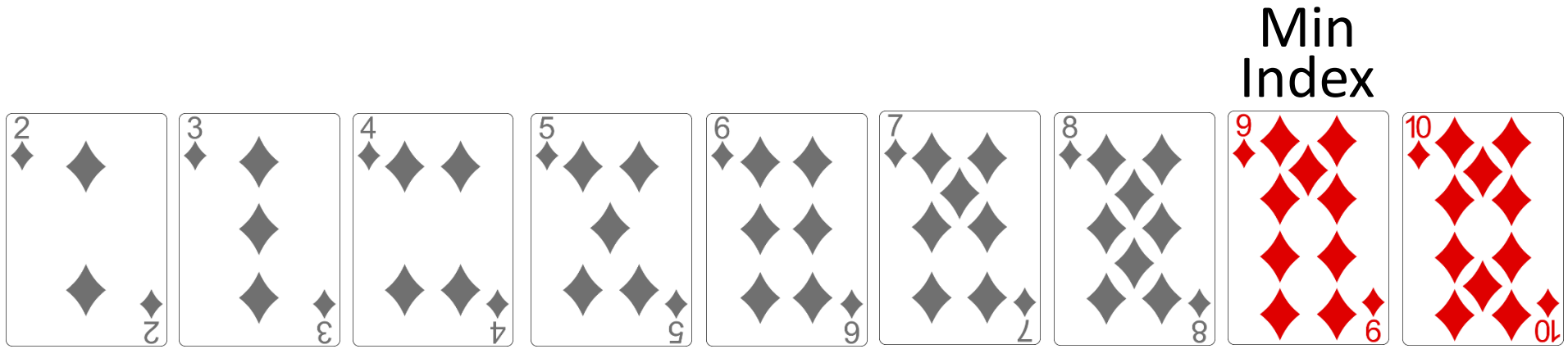


Min
Index

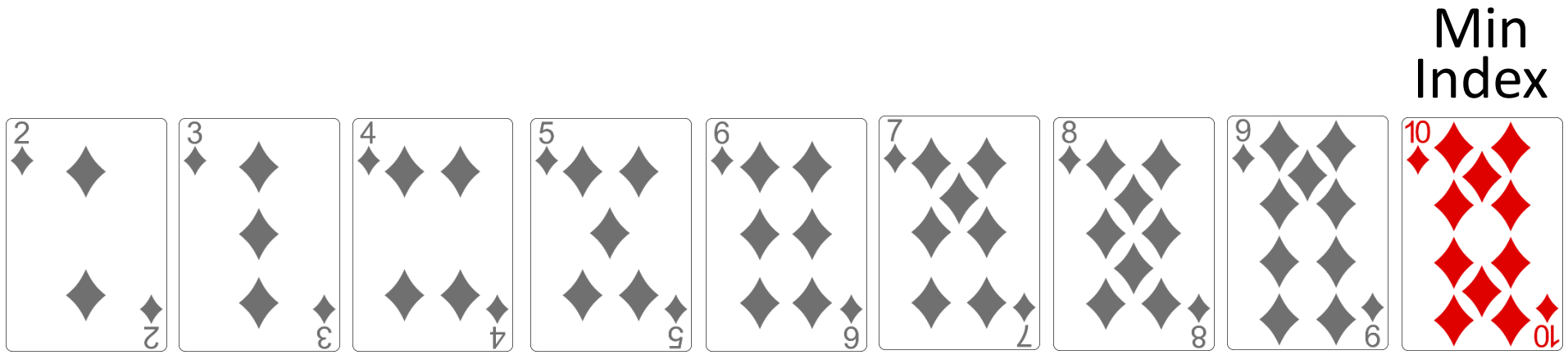
Selection Sort Visualization



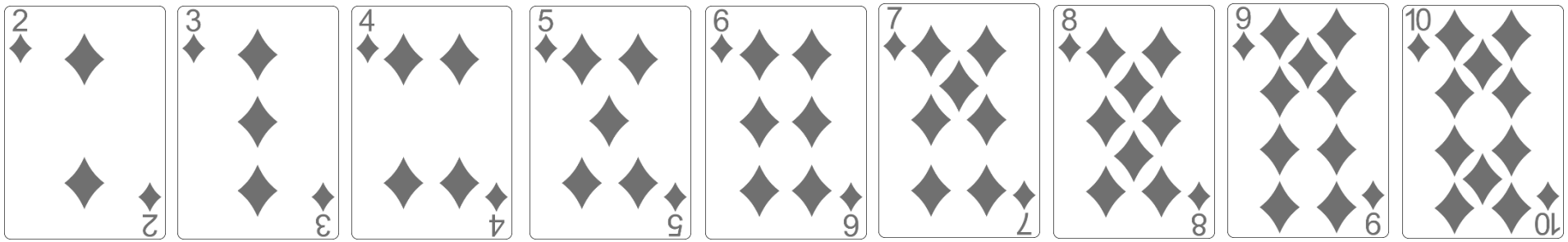
Selection Sort Visualization



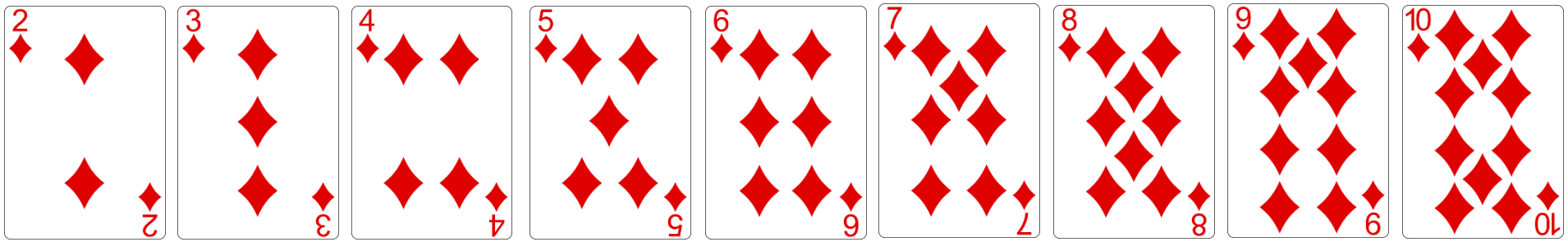
Selection Sort Visualization



Selection Sort Visualization



Selection Sort Visualization



Selection Sort Code

```
for (int index = 0; index < arr.length; index++) {
    int min = index;
    for (int curr = index + 1; curr < arr.length; curr++) {
        if (arr[curr] < arr[min]) {
            min = curr;
        }
    }
    swap(arr, index, min);
}
```

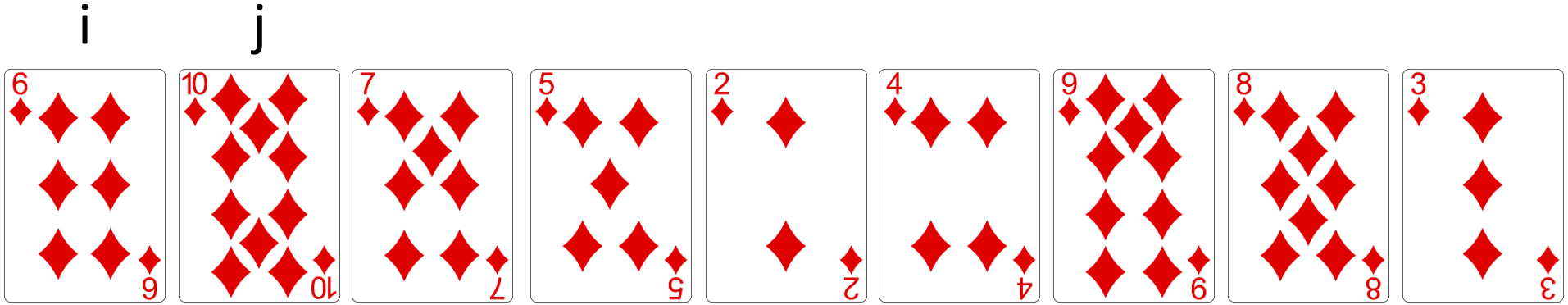
Comparison of Sorting Algorithms

Name	Best	Average	Worst	Memory	Stable	Method
Selection	n^2	n^2	n^2	1	No	Selection

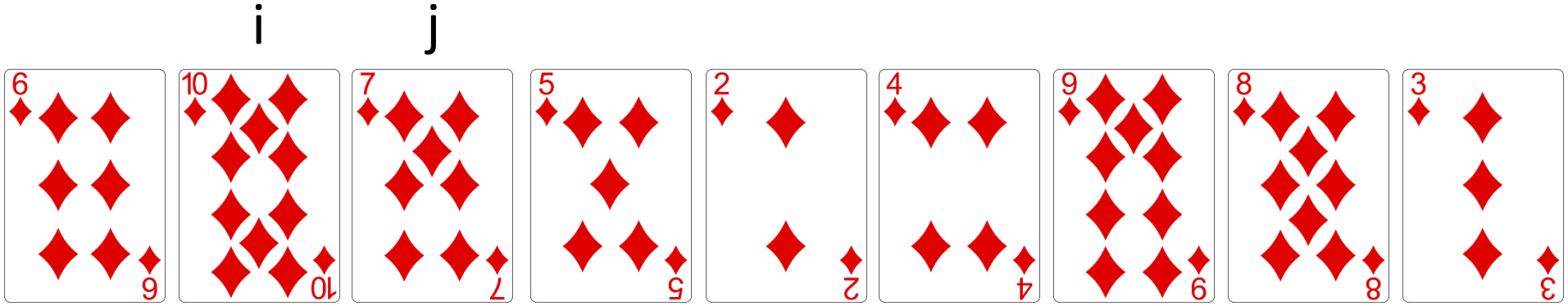
Bubble Sort

- **Bubble sort** – simple, but inefficient algorithm
- Swaps to neighbor elements when not in order until sorted
  - Memory: **O(1)**
  - Stable: Yes
  - Method: Exchanging

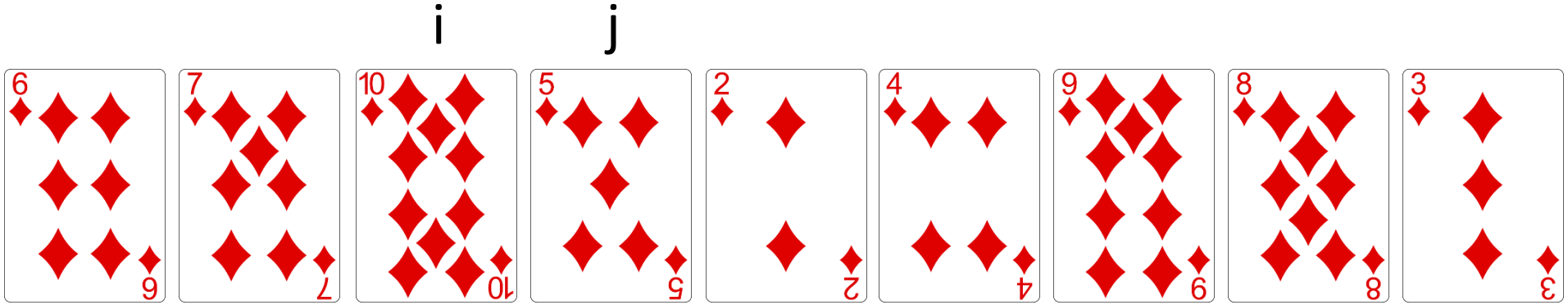
Bubble Sort Visualization



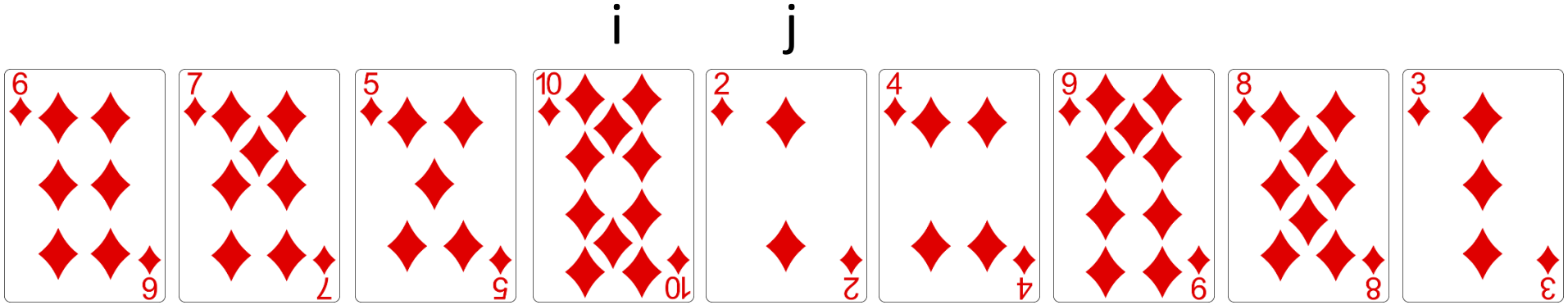
Bubble Sort Visualization



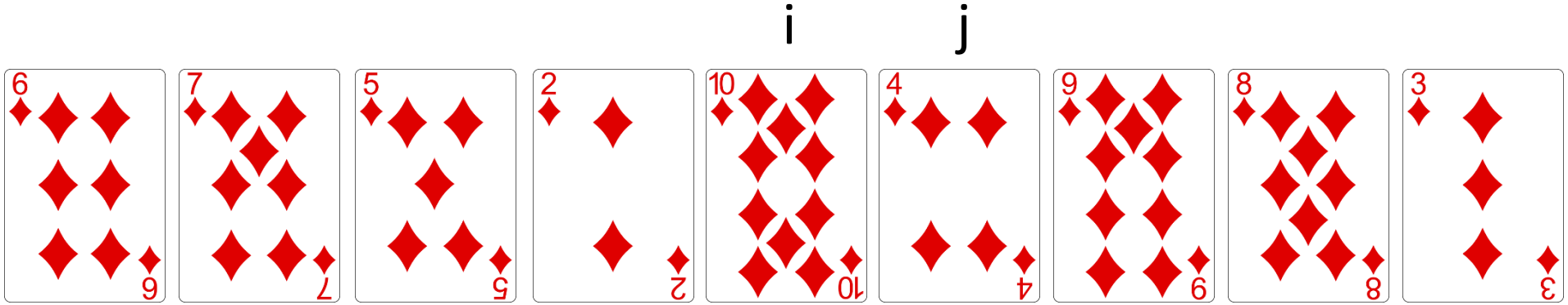
Bubble Sort Visualization



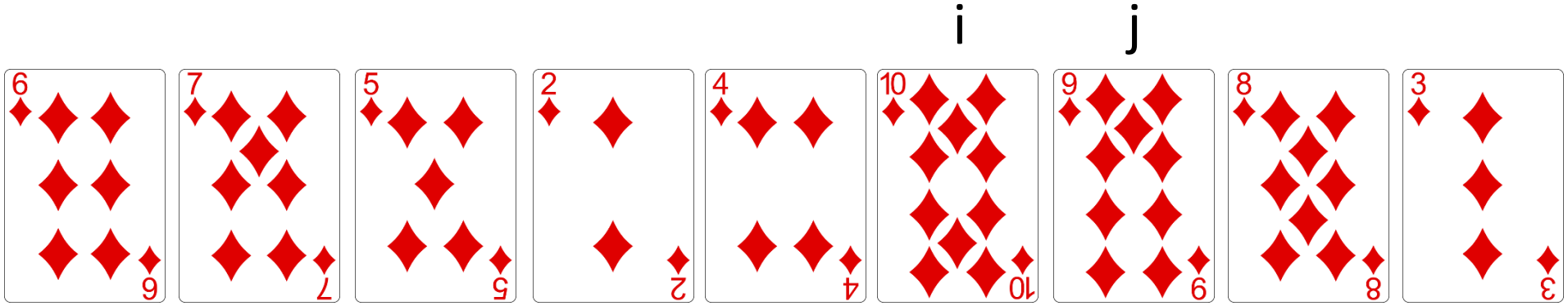
Bubble Sort Visualization



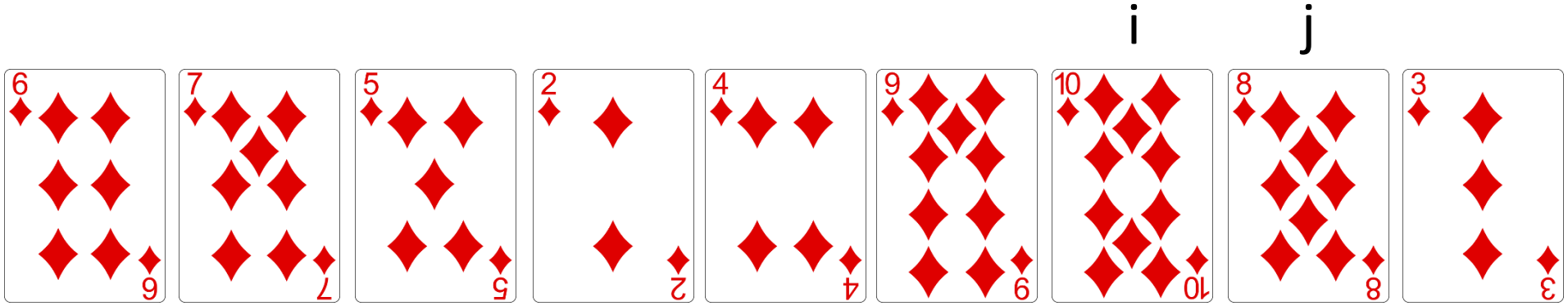
Bubble Sort Visualization



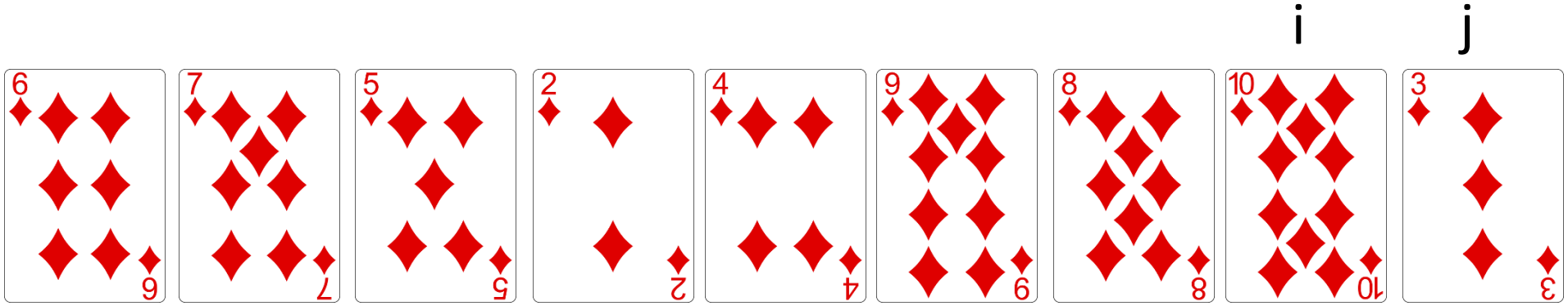
Bubble Sort Visualization



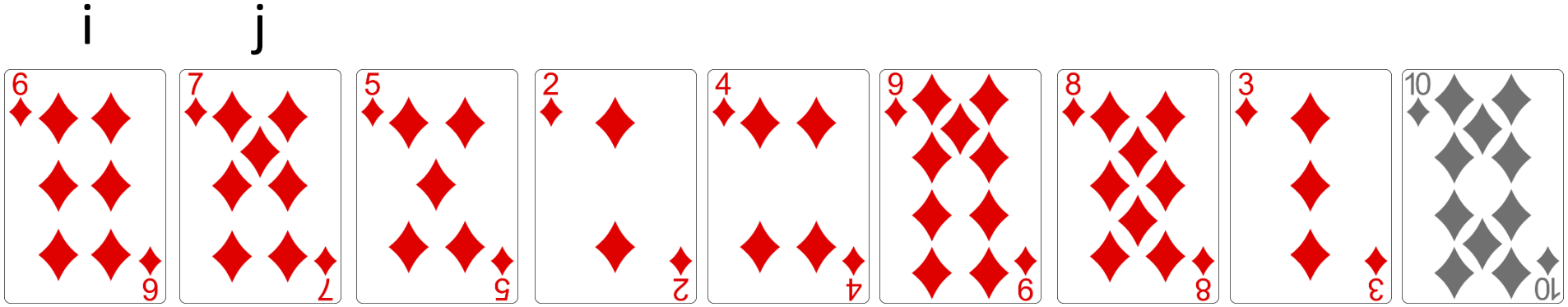
Bubble Sort Visualization



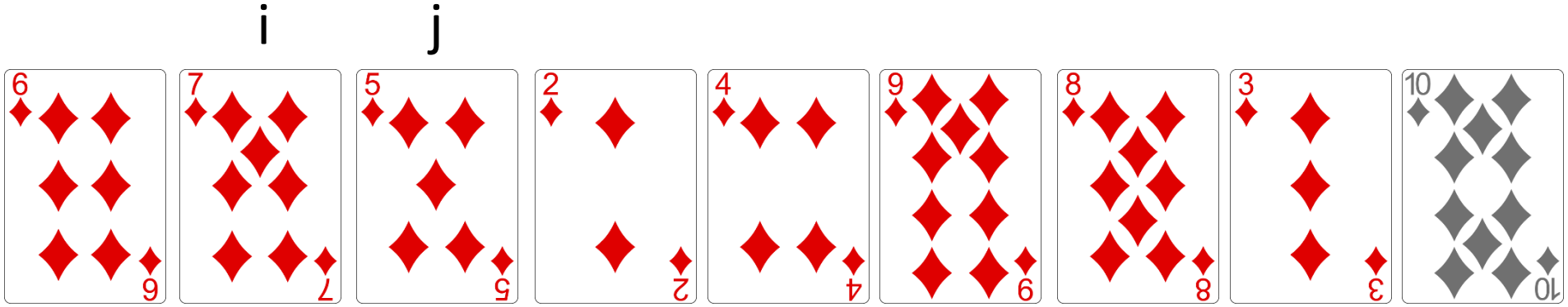
Bubble Sort Visualization



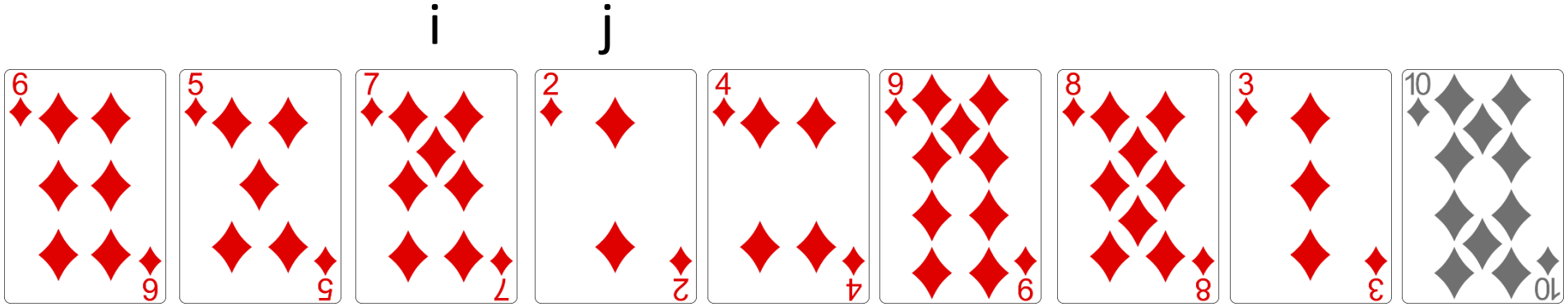
Bubble Sort Visualization



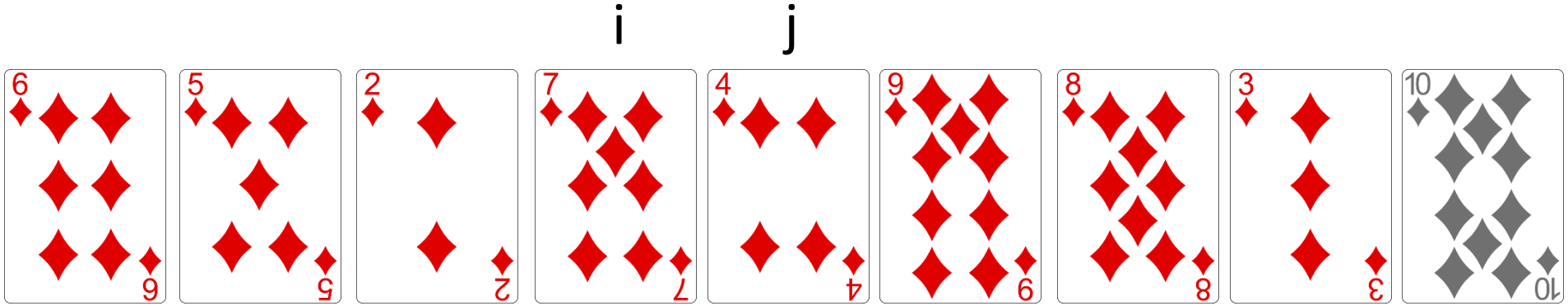
Bubble Sort Visualization



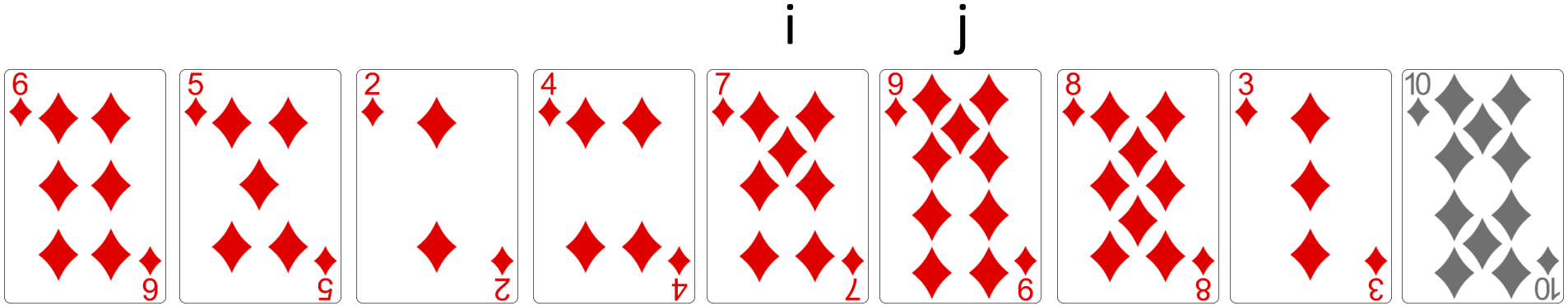
Bubble Sort Visualization



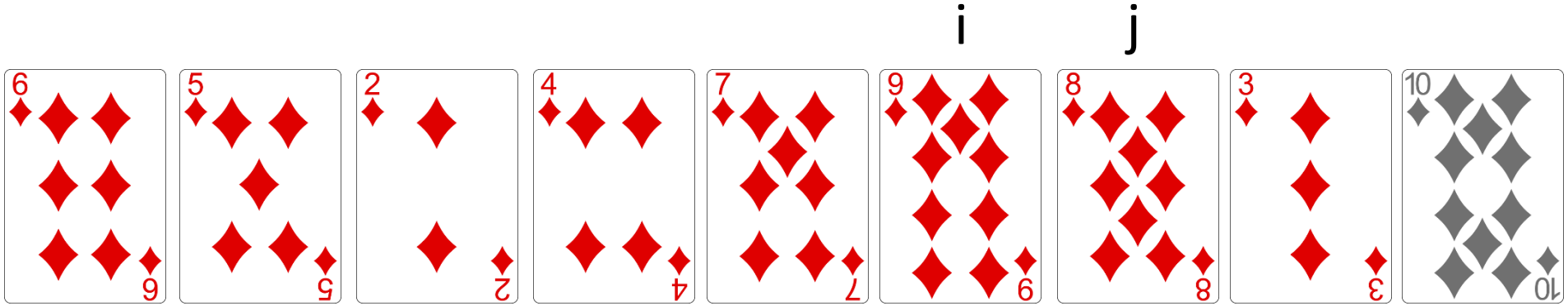
Bubble Sort Visualization



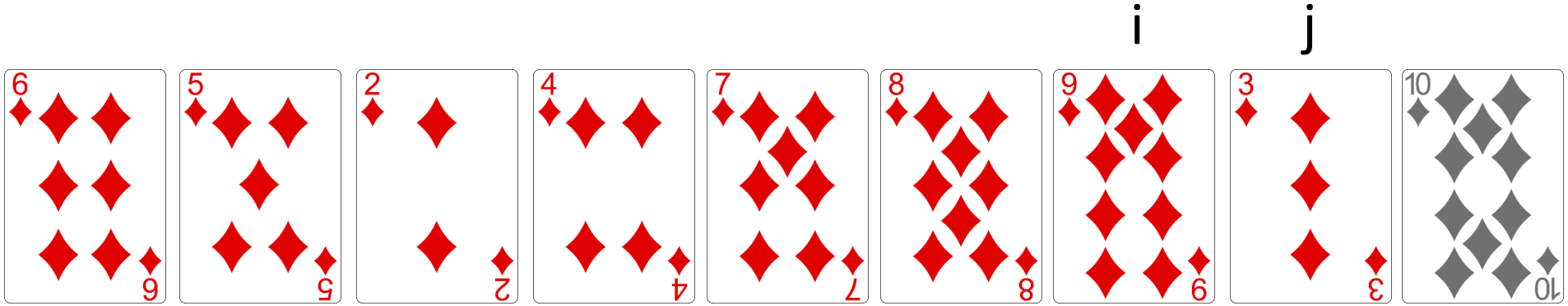
Bubble Sort Visualization



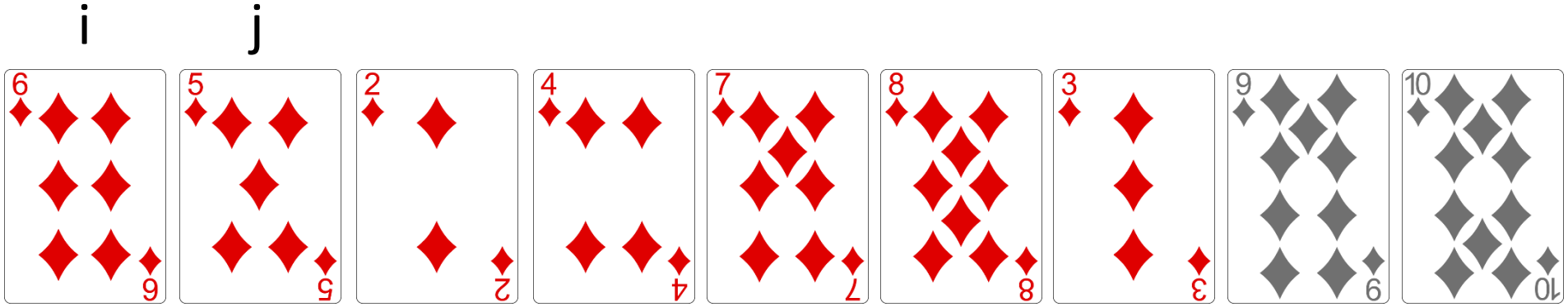
Bubble Sort Visualization



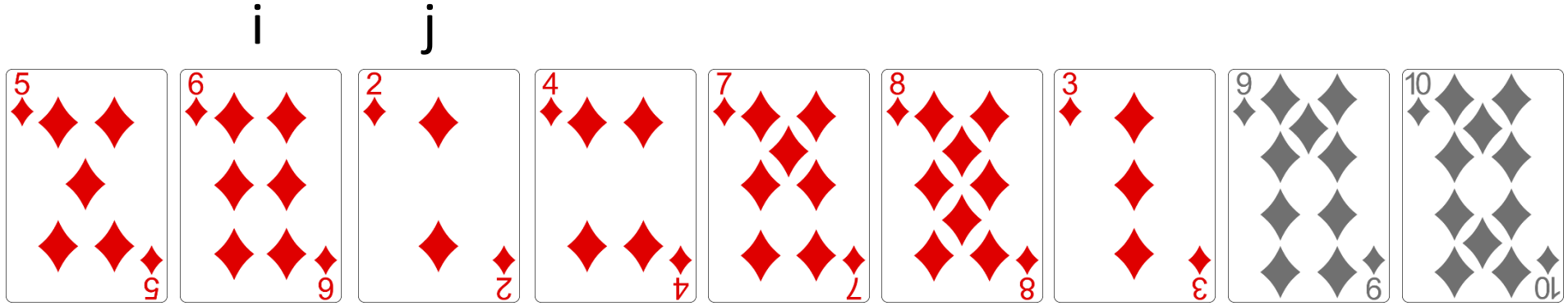
Bubble Sort Visualization



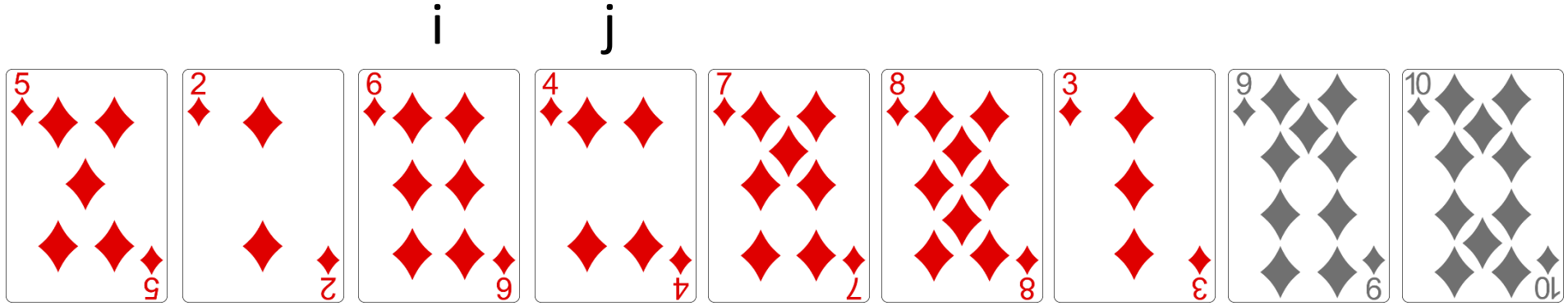
Bubble Sort Visualization



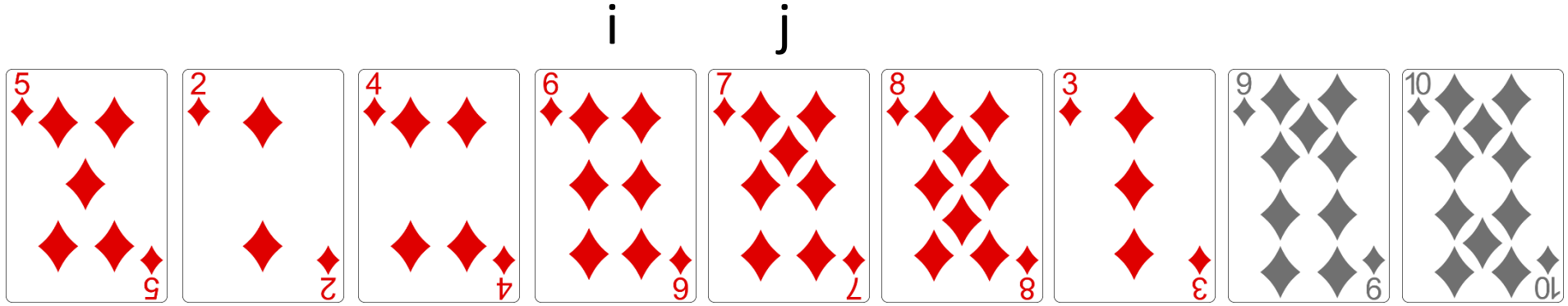
Bubble Sort Visualization



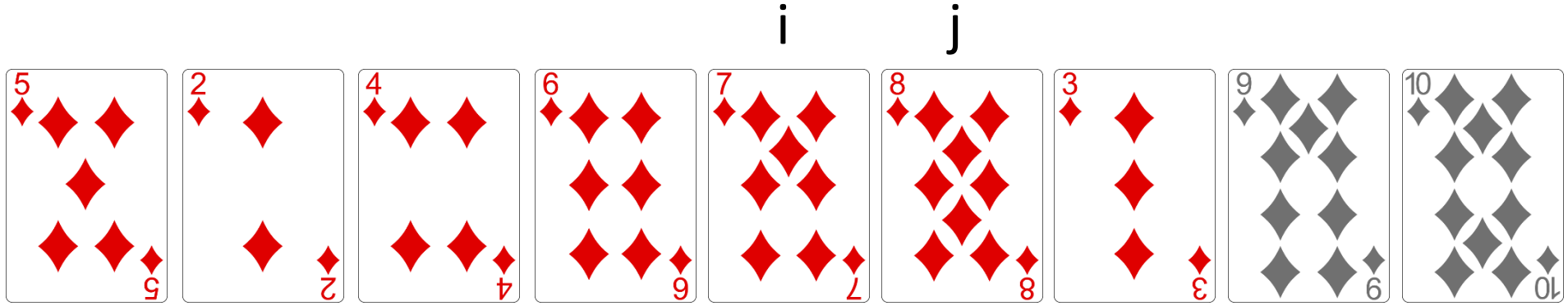
Bubble Sort Visualization



Bubble Sort Visualization

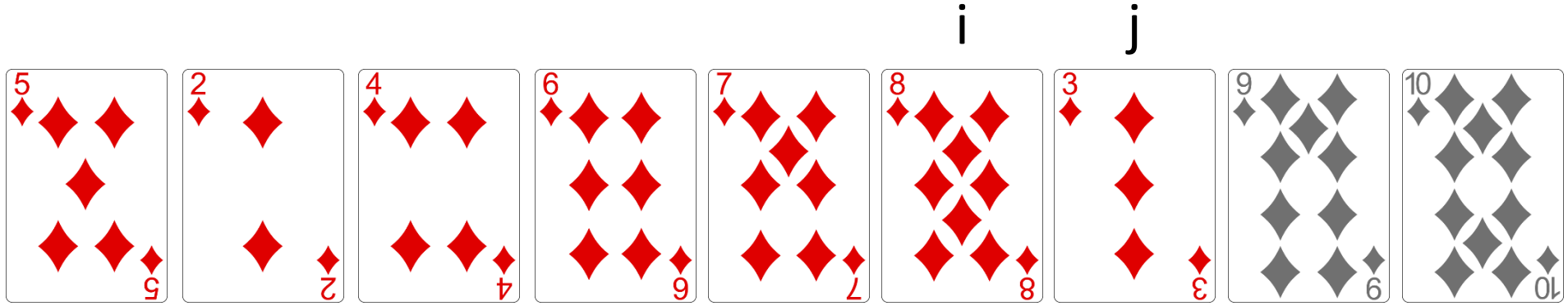


Bubble Sort Visualization

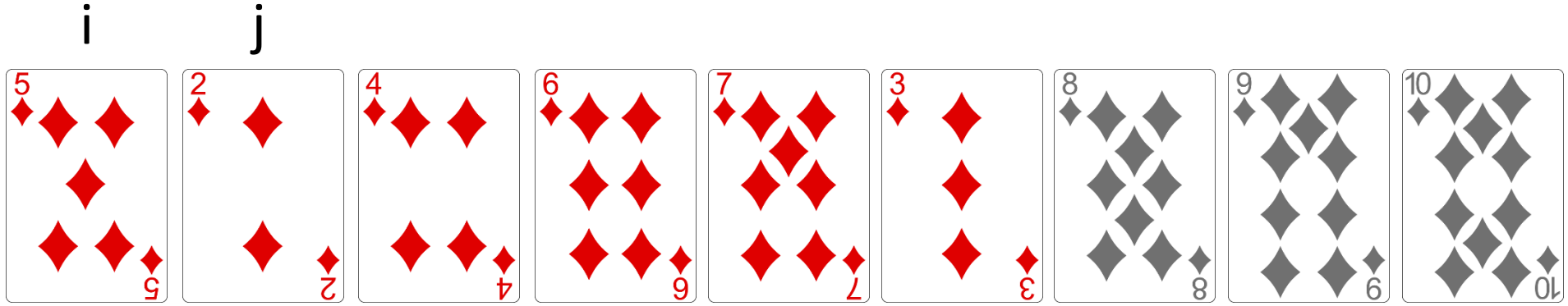


i j

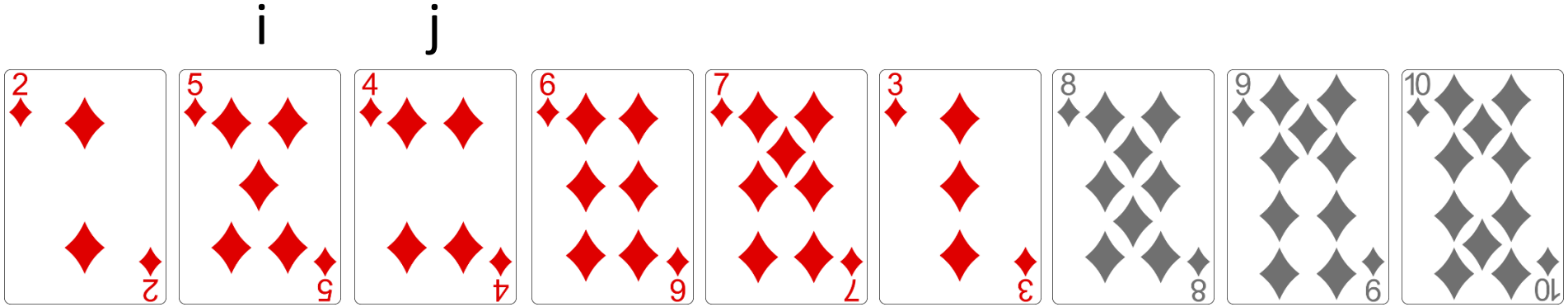
Bubble Sort Visualization



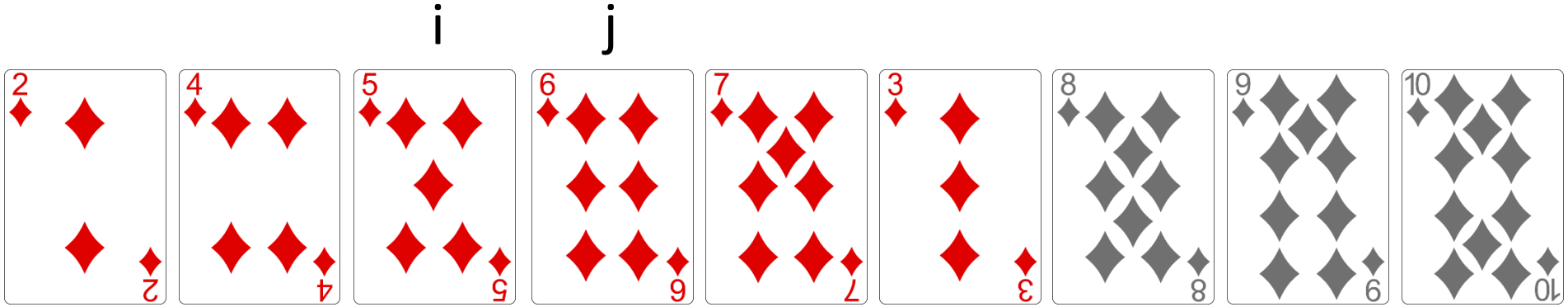
Bubble Sort Visualization



Bubble Sort Visualization

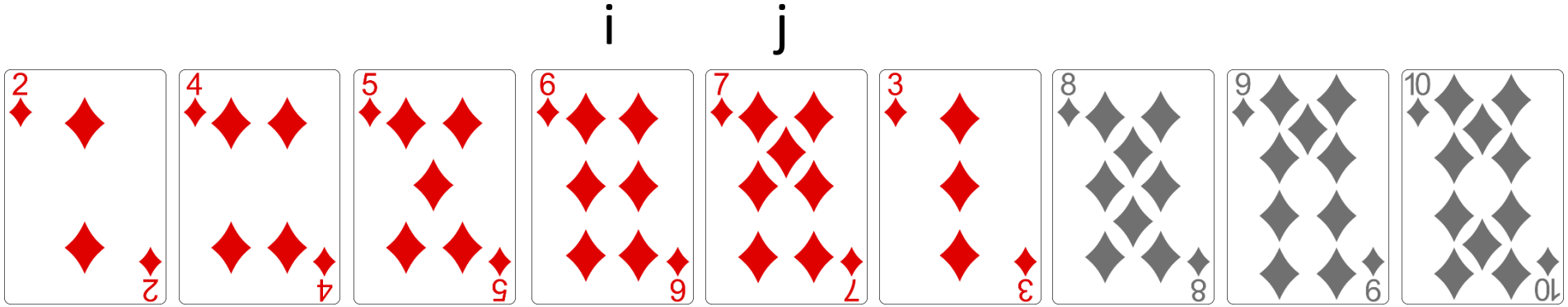


Bubble Sort Visualization



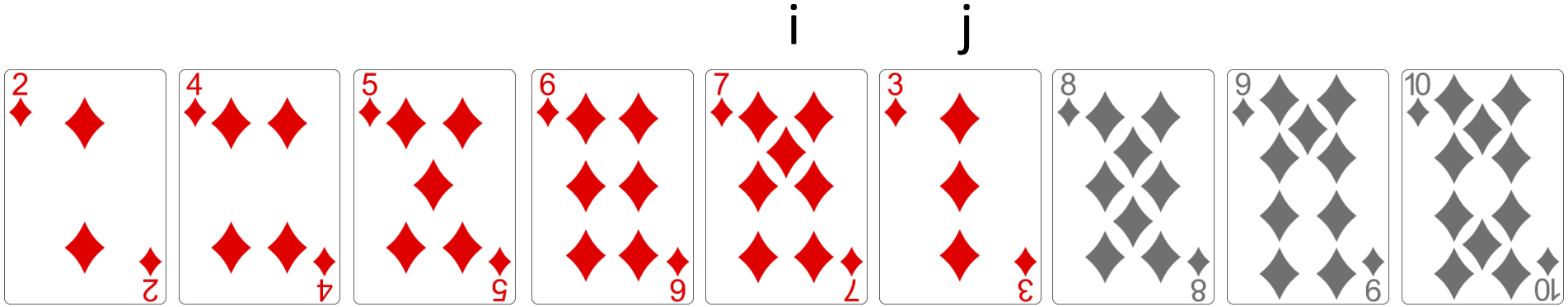
i j

Bubble Sort Visualization

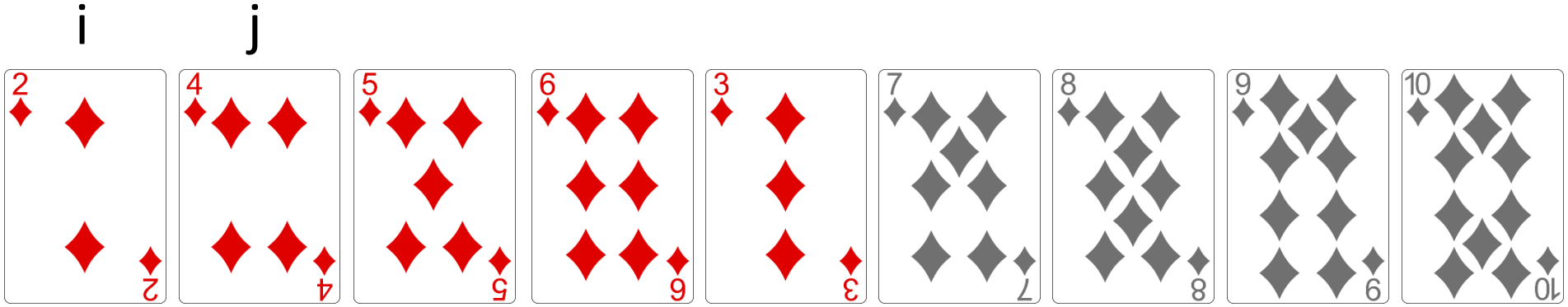


i j

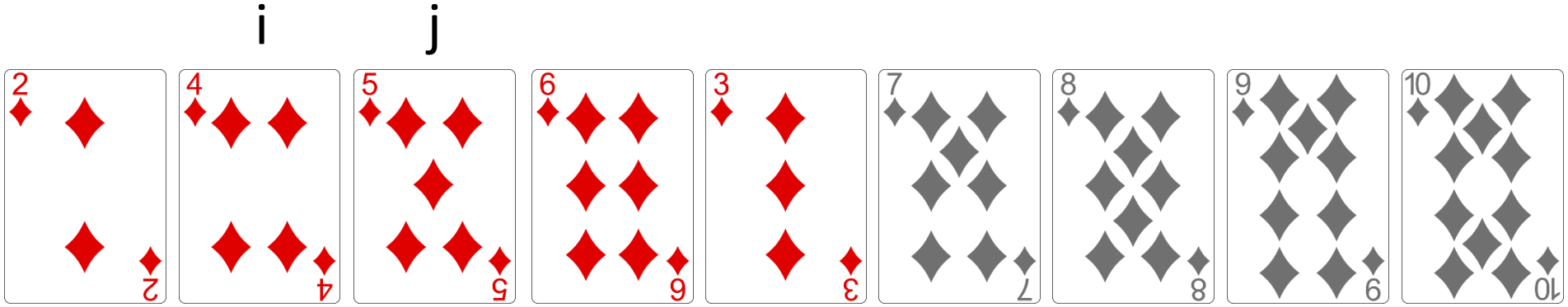
Bubble Sort Visualization



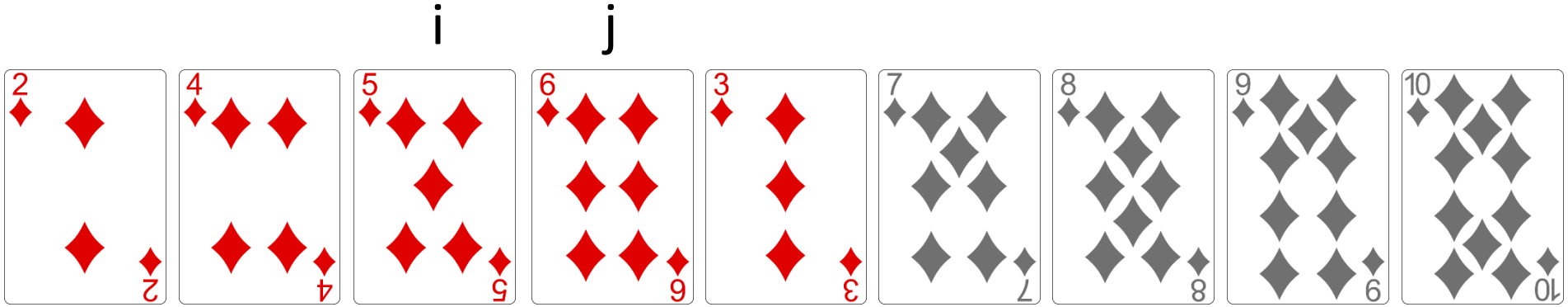
Bubble Sort Visualization



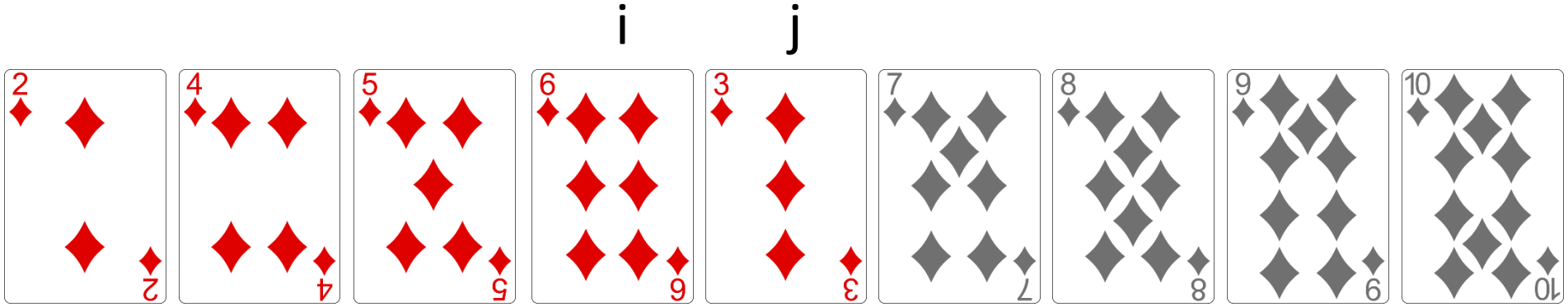
Bubble Sort Visualization



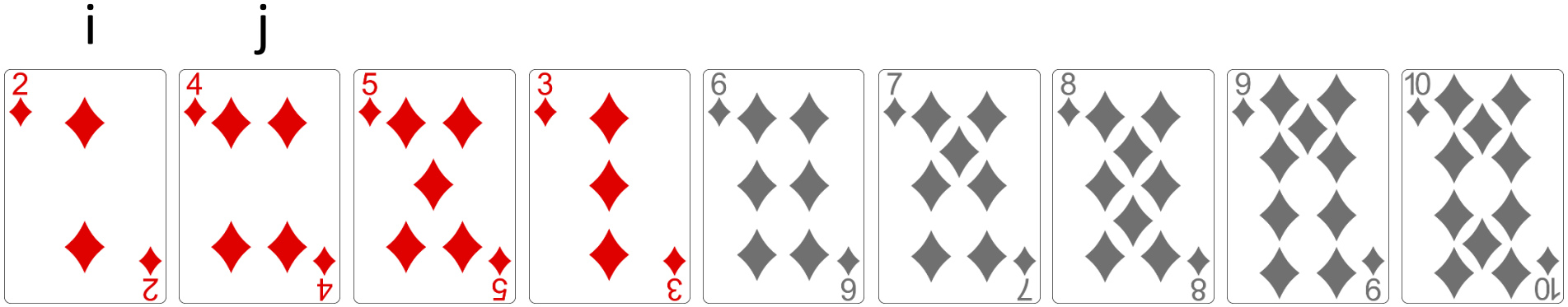
Bubble Sort Visualization



Bubble Sort Visualization

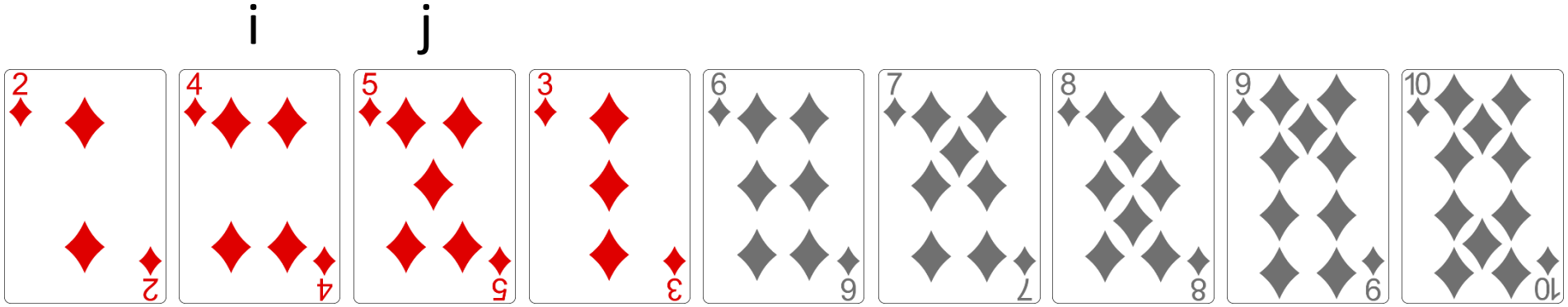


Bubble Sort Visualization

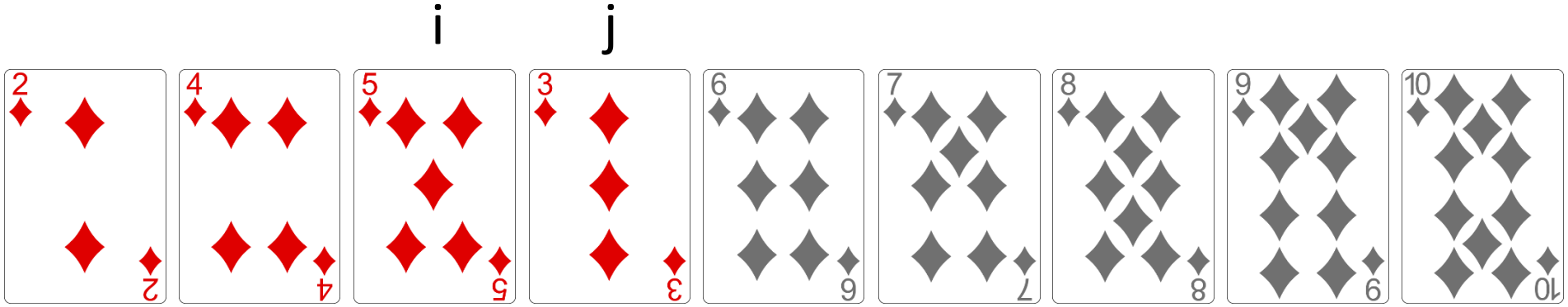


i j

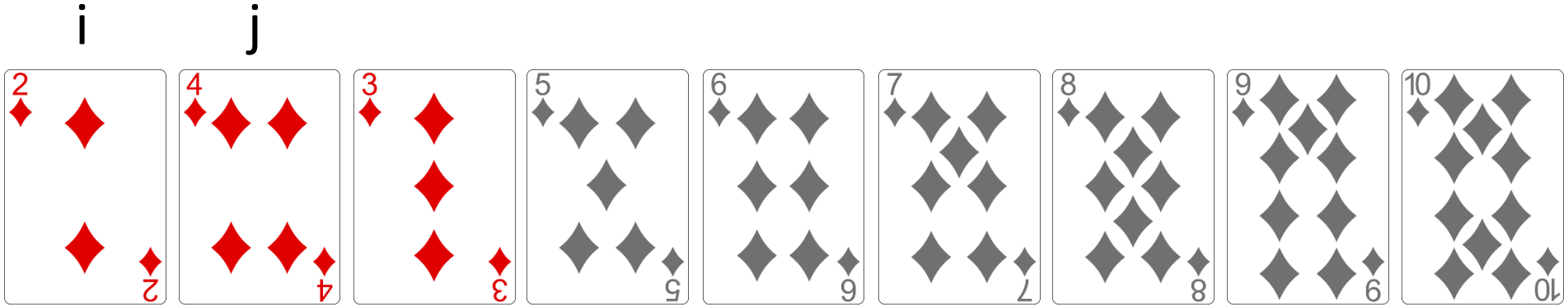
Bubble Sort Visualization



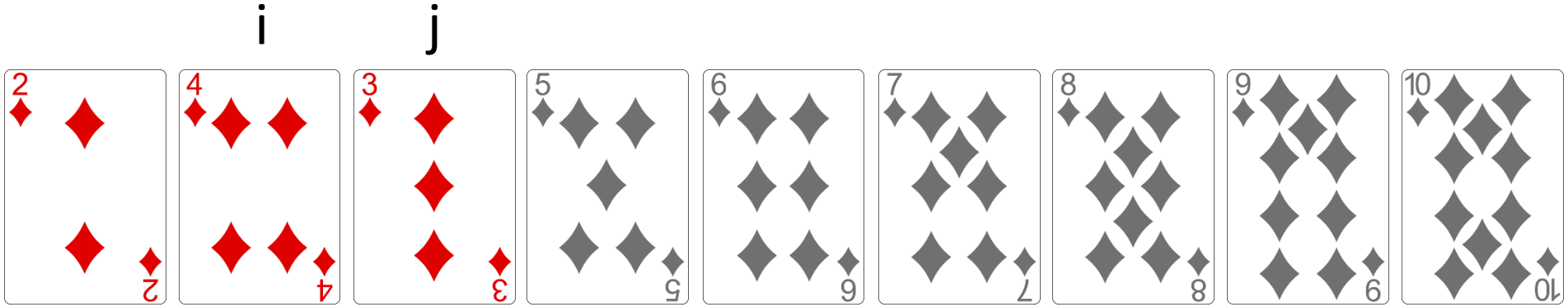
Bubble Sort Visualization



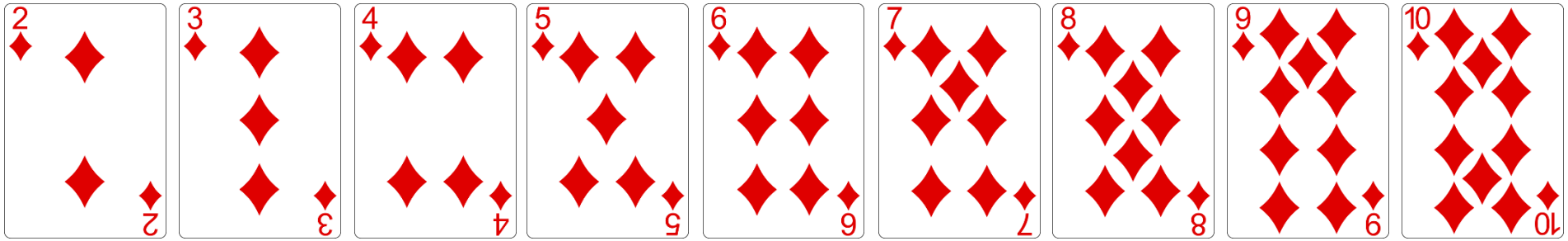
Bubble Sort Visualization



Bubble Sort Visualization



Bubble Sort Visualization



Bubble Sort Code

```
int[] numbers = {1, 3, 4, 2, 5, 6};
for (int i = 0; i < numbers.length; i++) {
  for (int j = i + 1; j < numbers.length - 1; j++) {
    if (numbers[i] > numbers[j]) {
      int tempNumber = numbers[i];
      numbers[i] = numbers[j];
      numbers[j] = tempNumber;
    }
  }
}
//TODO: Print numbers
```

Comparison of Sorting Algorithms

Name	Best	Average	Worst	Memory	Stable	Method
Selection	n^2	n^2	n^2	1	No	Selection
Bubble	n	n^2	n^2	1	Yes	Exchanging

Insertion Sort

- **Insertion Sort** – simple, but inefficient algorithm
  - Move the first unsorted element left to its place
  - Memory: O(1)
  - Stable: **Yes**
  - Method: **Insertion**

Insertion Sort

23 17 45 18 12 22

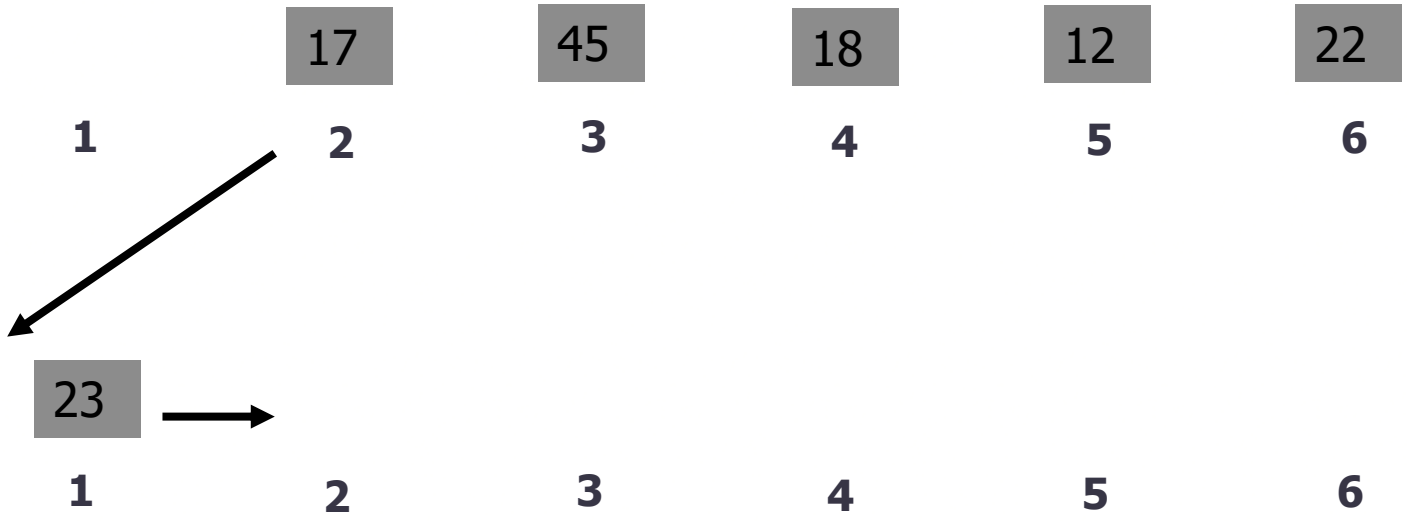
Insertion Sort

23	17	45	18	12	22
1	2	3	4	5	6

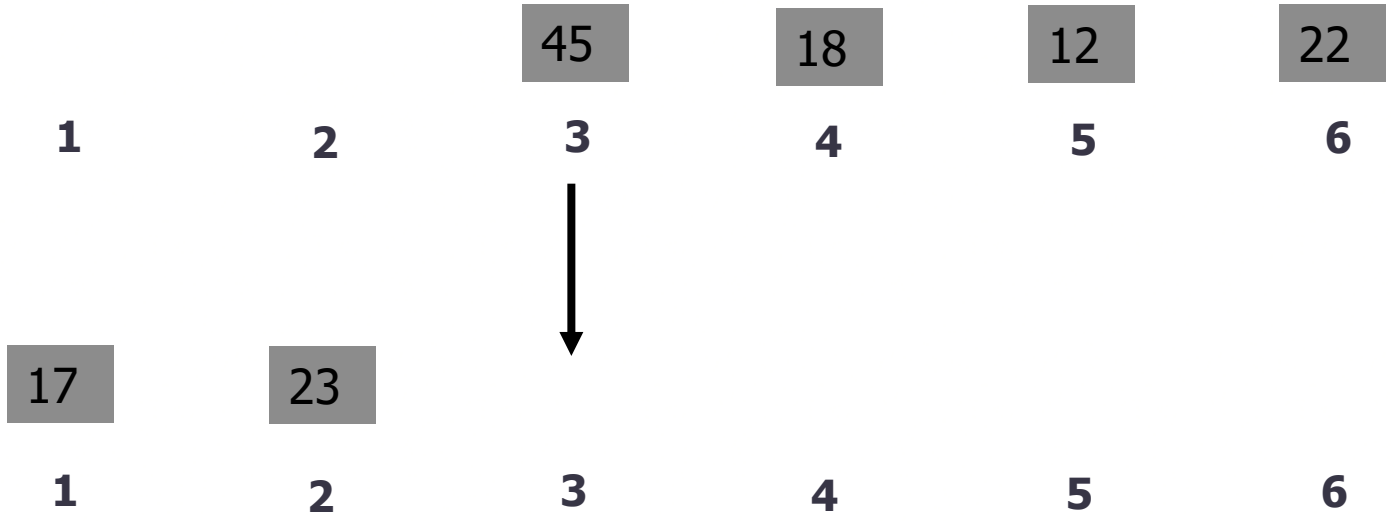
Insertion Sort

23	17	45	18	12	22
1	2	3	4	5	6
↓					
1	2	3	4	5	6

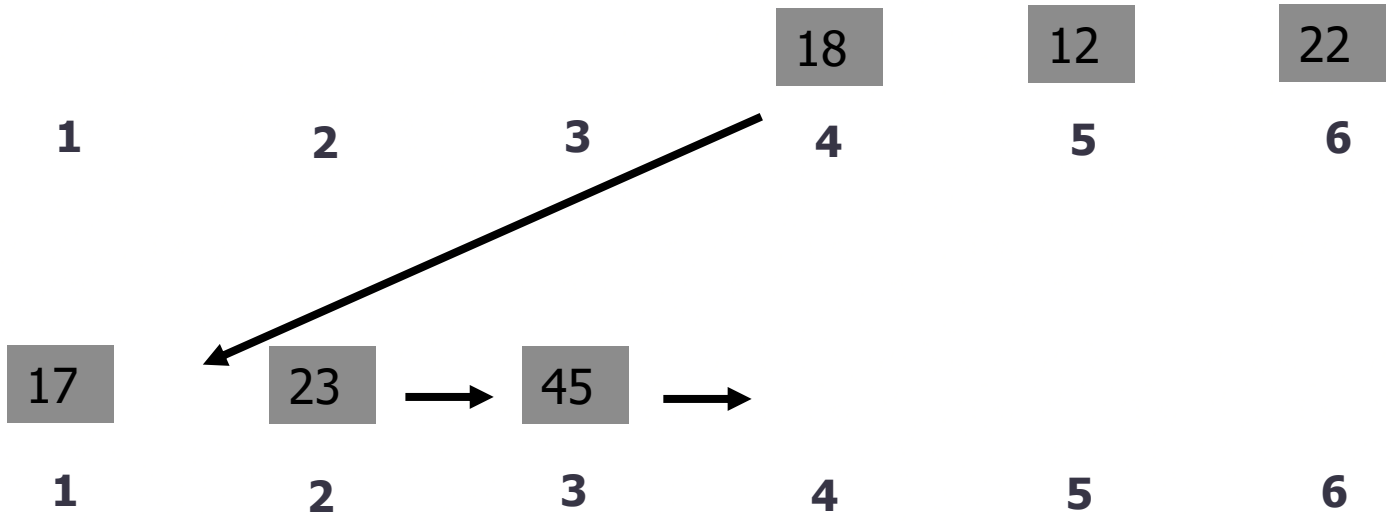
Insertion Sort



Insertion Sort



Insertion Sort



Insertion Sort

				12	22
1	2	3	4	5	6

17	18	23	45		
1	2	3	4	5	6

Insertion Sort

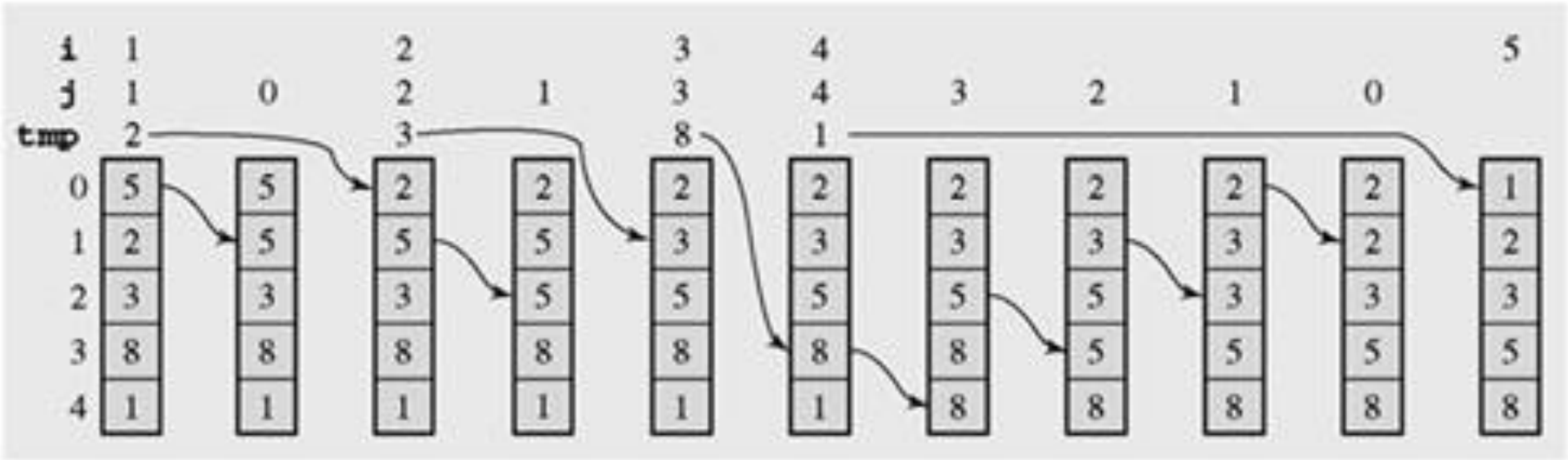
12	17	18	22	23	45
1	2	3	4	5	6

Insertion sort

- An insertion sort starts by considering the two first elements of the array data, which are data[0] and data[1]
- Next, the third element, data[2], is considered and inserted into its proper place (ascending)
- Its complexity is O(n2) and best case is O(n)

```
insertionsort(data[])
        for i=0 to data.length-1
                tmp=data[i];
                move all elements data[j] greater than tmp by one position;
                place tmp in its proper position;
```

Insertion sort example



The array [5 2 3 8 1] sorted by insertion sort

Insertion sort

```java
public static void main(String[] args) {
    int a[] = {5, 6, 1, 2, 4, 3, 7, 9, 8};
    insertionSort(a);
    for (int b : a) {
        System.out.print(b);
    }
    System.out.println("");
}
public static void insertionSort(int a[]){
    for (int i = 1; i < a.length; i++) {
        int temp = a[i];
        //find its correct place on the left.
        int j = i-1;
        while(j>=0 && a[j] > temp){
            //shift
            a[j+1] = a[j];
            j--;
        }
        a[++j] = temp;
    }
}
```

Comparison of Sorting Algorithms

Name	Best	Average	Worst	Memory	Stable	Method
Selection	n^2	n^2	n^2	1	No	Selection
Bubble	n	n^2	n^2	1	Yes	Exchanging
Insertion	n	n^2	n^2	1	Yes	Insertion

Shuffling

- **Shuffling** == randomizing the order of items in a collection
  - Generate a random permutation
- *The generation of random numbers is too important to be left to chance.* —Robert R. Coveyou

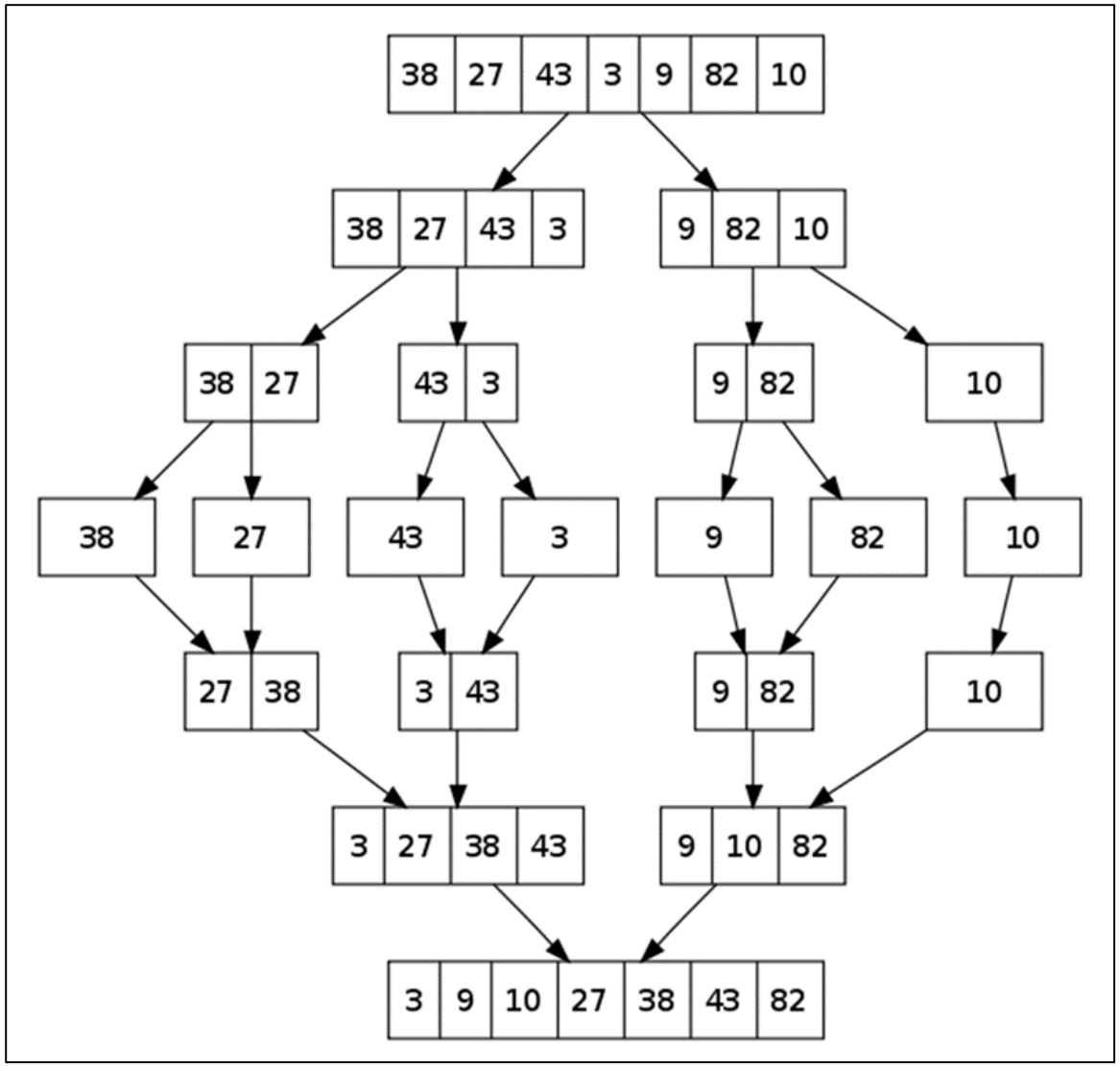
Fisher–Yates Shuffle Algorithm

```
public static <T> void shuffle(T[] source) {
    Random rand = new Random();
    for (int i = 0; i < source.length; i++) {
        // Exchange array[i] with random element in array[i … n-1]
        int r = i + rand.nextInt(source.length - i);
        T temp = source[i];
        source[i] = source[r];
        source[r] = temp;
    }
}
```

Merge Sort

- **Merge sort** is efficient sorting algorithm
- Divide the list into sub-lists (typically 2 sub-lists)
  1. Sort each sub-list (recursively call merge-sort)
  2. Merge the sorted sub-lists into a single list
- Best, average and worst case: **O(n*log(n))**
- Memory:
  - Typically
  - With in-place merge can be
- Highly parallelizable on multiple cores / machines → up to **O(log(n))**

Merge Sort: Hot It Works?



QuickSort

- **QuickSort** – efficient sorting algorithm
  - Choose a pivot; move smaller elements left & larger right; sort left & right
  - Best & average case: **O(n*log(n))**; Worst: **O(n2)**
  - Memory: **O(log(n))** stack space (for recursion)
  - Stable: **Depends**
  - Method: **Partitioning**

Counting Sort

- **Counting sort** is a very efficient sorting algorithm
- Sorts small integers by counting their occurrences
  - Not a comparison-based sort
  - Best, average and worst case: **O(n + k)**
    - **k** is the range of numbers to be sorted
    - E.g. [-1000 ... 1000] → **k** = 2001
- Memory: **O(n + k)** Space: **O(k)**
- Stable: Yes
- Method: Counting

Bucket Sort

- **Bucket sort** partitions an array into a number of buckets
  - Each bucket is then sorted individually with a different algorithm
  - Not a comparison-based sort
- Average case: **O(n + k)**
  - **k** = the number of buckets
- Worst case: **O(n * log n)**
- Stable: Yes (depends on algorithm)
- Memory: **O(n)** – **k** buckets holding n elements totally

Comparison of Sorting Algorithms

Name	Best	Average	Worst	Memory	Stable	Method
Selection	n^2	n^2	n^2	1	No	Selection
Bubble	n	n^2	n^2	1	Yes	Exchanging
Insertion	n	n^2	n^2	1	Yes	Insertion
Quick	n * log(n)	n * log(n)	n^2	1	Depends	Partitioning
Merge	n * log(n)	n * log(n)	n * log(n)	1 (or n)	Yes	Merging
Heap	n * log(n)	n * log(n)	n * log(n)	1	No	Selection
Bogo	n	n * n!	n * n!	1	No	Luck

Summary

- **Searching** algorithms
  - Binary Search, Interpolation Search
- **Slow** sorting algorithms:
  - Selection sort, Bubble sort, Insertion sort
- **Fast** sorting algorithms:
  - Quick sort, Merge sort, etc.
  - How to choose the most appropriate algorithm?
    - http://stackoverflow.com/a/1934004